

LAPORAN MODUL 11 PRAKTIKUM ALGORITMA DAN PEMROGRAMAN LANJUTAN



Disusun oleh :

Nama : Irvan Cahya Nugraha
NIM : 245410034
Kelas : Informatika 1

**PROGRAM STUDI INFORMATIKA
PROGRAM SARJANA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS TEKNOLOGI DIGITAL INDONESIA
YOGYAKARTA
2024 / 2025**

MODUL 11

REKURSIF : definisi, cara kerja

A. TUJUAN PRAKTIKUM

1. Mahasiswa dapat memahami dan menyelesaikan kasus dengan rekursif

B. DASAR TEORI

Program-program yang telah kita bahas sejauh ini umumnya disusun sebagai method yang memanggil satu sama lain secara hierarkis. Untuk beberapa masalah, ada baiknya memanggil method itu sendiri. Method yang dikenal sebagai method rekursif. Method rekursif dapat memanggil dirinya baik secara langsung atau tidak langsung melalui method lain.

Konsep rekursif.

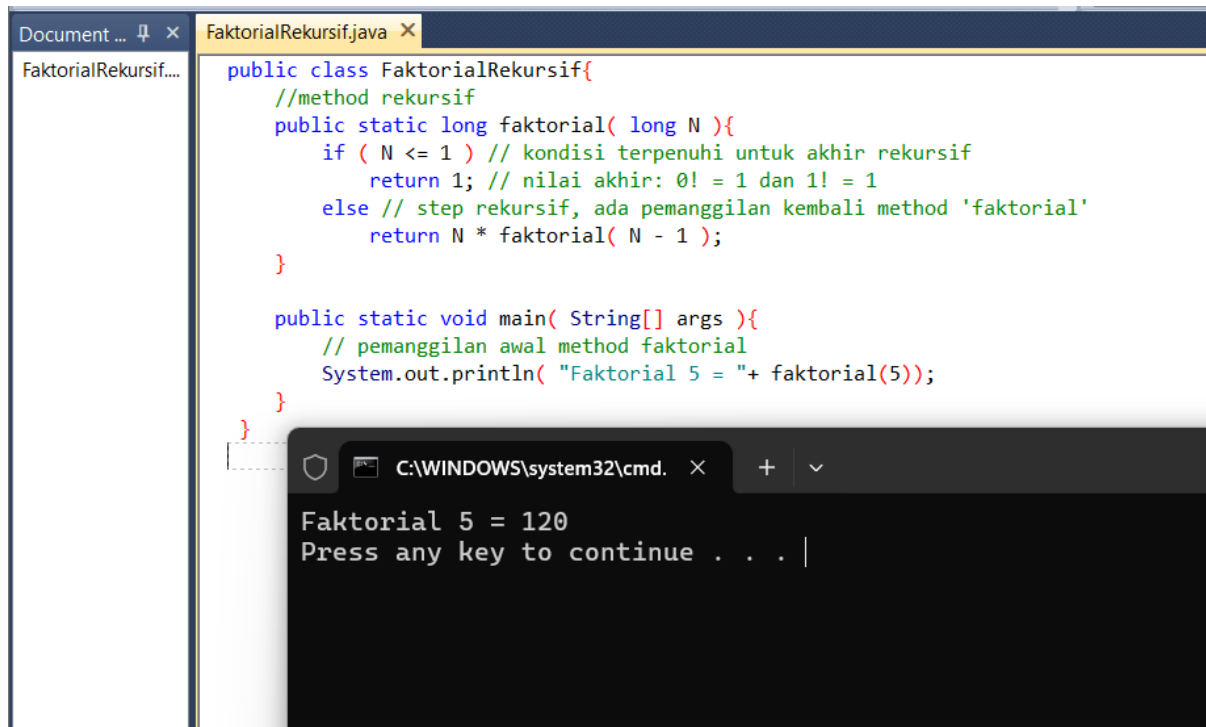
Pendekatan pemecahan masalah rekursif memiliki sejumlah elemen yang sama. Ketika method rekursif dipanggil untuk memecahkan masalah, sebenarnya hanya mampu menyelesaikan kasus yang paling sederhana, atau kasus dasar. Jika method ini disebut dengan kasus dasar, method itu mengembalikan hasil. Jika method ini disebut dengan masalah yang lebih kompleks, method tersebut biasanya membagi masalah menjadi dua bagian konseptual — bagian yang diketahui cara melakukannya dan bagian yang tidak diketahui bagaimana melakukannya. Untuk membuat rekursi menjadi layak, bagian yang terakhir harus menyerupai masalah aslinya, tetapi versi yang sedikit lebih sederhana atau lebih kecil. Karena masalah baru ini terlihat seperti masalah asli, method ini memanggil salinan baru dari dirinya untuk bekerja pada masalah yang lebih kecil — ini disebut sebagai panggilan rekursif dan juga disebut langkah rekursi. Langkah rekursi biasanya mencakup pernyataan pengembalian, karena hasilnya akan digabungkan dengan bagian dari masalah yang diketahui method untuk menyelesaikan untuk membentuk hasil yang akan diteruskan kembali ke pemanggil asli. Langkah rekursi dijalankan sementara pemanggilan method asli masih aktif (mis., Belum selesai dieksekusi). Ini dapat menghasilkan lebih banyak panggilan rekursif karena method ini membagi setiap sub-masalah baru menjadi dua bagian konseptual. Agar rekursi akhirnya berakhir, setiap kali method memanggil dirinya sendiri dengan versi yang lebih sederhana dari masalah aslinya, urutan masalah yang lebih kecil dan lebih kecil harus menyatu pada kasus dasar. Ketika method mengenali kasus dasar, itu mengembalikan hasil ke salinan method sebelumnya. Urutan pengembalian terjadi sampai pemanggilan method asli mengembalikan hasil akhir ke pemanggil.

Method rekursif dapat memanggil method lain, yang pada gilirannya membuat panggilan kembali ke method rekursif. Ini dikenal sebagai panggilan rekursif tidak langsung atau rekursi tidak langsung. Misalnya, method A memanggil method B, yang membuat panggilan kembali ke method A. Ini masih rekursi, karena panggilan kedua ke method A dilakukan saat panggilan pertama ke method A aktif — yaitu panggilan pertama ke method A belum selesai dieksekusi (karena sedang menunggu method B untuk mengembalikan hasilnya) dan belum kembali ke pemanggil asli method A. Untuk lebih memahami konsep rekursi, mari kita lihat contoh yang cukup akrab bagi pengguna komputer — definisi rekursif direktori pada komputer. Komputer biasanya menyimpan file terkait dalam direktori. Direktori dapat kosong, dapat berisi file dan / atau dapat berisi direktori lain (biasanya disebut sebagai subdirektori). Masing-masing subdirektori ini, pada gilirannya, dapat juga berisi file dan direktori. Jika kita ingin membuat daftar setiap file dalam direktori (termasuk semua file dalam subdirektori direktori),

kita perlu membuat method yang pertama-tama mendaftarkan file direktori awal, kemudian membuat panggilan rekursif untuk membuat daftar file di masing-masing subdirektori direktori itu. Kasus dasar terjadi ketika direktori tercapai yang tidak mengandung subdirektori. Pada titik ini, semua file dalam direktori asli telah terdaftar dan tidak perlu rekursi lebih lanjut. (Sumber : Deitel, Deitel edisi 9)

C. PEMBAHASAN LISTING PRAKTIK

Praktik 1



```
Document ... 1 x FaktorialRekursif.java x
FaktorialRekursif...
public class FaktorialRekursif{
    //method rekursif
    public static long faktorial( long N ){
        if ( N <= 1 ) // kondisi terpenuhi untuk akhir rekursif
            return 1; // nilai akhir: 0! = 1 dan 1! = 1
        else // step rekursif, ada pemanggilan kembali method 'faktorial'
            return N * faktorial( N - 1 );
    }

    public static void main( String[] args ){
        // pemanggilan awal method faktorial
        System.out.println( "Faktorial 5 = "+ faktorial(5));
    }
}
```

```
C:\WINDOWS\system32\cmd. x + v
Faktorial 5 = 120
Press any key to continue . . . |
```

Pembahasan : Pada program FaktorialRekursif, terdapat satu method utama bernama faktorial yang bertipe long dan menerima parameter long N. Method ini adalah method rekursif karena di dalamnya terjadi pemanggilan terhadap dirinya sendiri. Pada baris `if ( N <= 1 )`, kondisi ini menjadi basis rekursi atau base case, yaitu saat N bernilai 1 atau kurang, maka method akan mengembalikan nilai 1 sebagai hasil faktorial dari 1 dan juga 0. Jika N lebih dari 1, maka dijalankan langkah rekursi pada `return N * faktorial(N - 1)`, di mana method faktorial memanggil dirinya dengan N - 1 hingga akhirnya mencapai basis rekursi. Ini menghasilkan proses pemanggilan berulang hingga kasus terkecil tercapai, lalu nilai dikalikan satu per satu dalam urutan kembali (unwinding) ke pemanggilan awal.

Praktik 2

```
Document ... x FibonacciRekursif.java x FaktorialRekursif.java
FaktorialRekursif...
FibonacciRekursi...

public class FibonacciRekursif{
    //method rekursif
    public static long fibonacci( long N ){
        if (( N == 0 ) || ( N == 1 )) // kondisi terpenuhi untuk akhir rekursif
            return N; // nilai akhir : N=0 atau N=1
        else // step rekursif, ada pemanggilan kembali method 'fibonacci'
            return fibonacci(N-1) + fibonacci(N-2);
    }

    public static void main( String[] args ){
        System.out.println("Deret fibonacci dengan N = 3");
        for (int i=0; i<=3;i++)
            // pemanggilan awal method fibonacci
            if (i<3)
                System.out.print(fibonacci(i)+", ");
            else
                System.out.println(fibonacci(i));
    }
}

C:\WINDOWS\system32\cmd. x + v
Deret fibonacci dengan N = 3
0, 1, 1, 2
Press any key to continue . . . |
```

Pembahasan : Program FibonacciRekursif menggunakan method fibonacci yang juga bertipe long dan menerima parameter long N. Basis rekursinya adalah `if (( N == 0 ) || ( N == 1 ))`, di mana jika N bernilai 0 atau 1 maka akan langsung dikembalikan nilainya karena itu adalah elemen dasar dari deret Fibonacci. Langkah rekursinya adalah `return fibonacci(N-1) + fibonacci(N-2)`, yang mencerminkan rumus dasar Fibonacci bahwa $F(N) = F(N-1) + F(N-2)$. Pada method main, terdapat perulangan for untuk menampilkan nilai fibonacci dari $i=0$ hingga $i=3$. Setiap iterasi memanggil method `fibonacci(i)` dan menampilkannya dengan pemisah koma kecuali pada nilai terakhir.

Praktik 3

```
Document ... x Prak3.java FibonacciRekursif.java FaktorialRekursif.java
FaktorialRekursif...
FibonacciRekursi...
Prak3.java

public class Prak3 {

    // Method rekursif untuk menghitung faktorial
    public static long faktorial(int n) {
        if (n == 1)
            return 1;
        else
            return n * faktorial(n - 1);
    }

    public static void main(String[] args) {
        int N = 5;
        System.out.println("Faktorial dari 1 sampai " + N + ":");

        for (int i = 1; i <= N; i++) {
            long hasil = faktorial(i);
            System.out.println(i + "! = " + hasil);
        }
    }
}

C:\WINDOWS\system32\cmd. x + v

Faktorial dari 1 sampai 5:
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
Press any key to continue . . . |
```

Pembahasan : Modifikasi dari faktorial sebelumnya adalah menampilkan semua hasil faktorial dari 1 hingga N. Pada praktik ini, kita perlu membuat perulangan dari 1 sampai N, dan untuk setiap nilai i, method faktorial(i) dipanggil lalu dicetak. Jadi di method main, kita menggunakan perulangan for untuk i dari 1 sampai 5, kemudian mencetak hasil faktorial(i) di setiap iterasi. Method faktorial tetap sama, dengan basis $N \leq 1$ mengembalikan 1 dan langkah rekursi $\text{return } N * \text{faktorial}(N - 1)$.

LATIHAN

Latihan 1

```
Document ... x Lat1.java Prak3.java FibonacciRekursif.java FaktorialRekursif.java
FaktorialRekursif...
FibonacciRekursi...
Lat1.java
Prak3.java

public class Lat1 {

    public static int pangkat(int A, int B) {
        if (B == 0) {
            return 1; // Basis: A^0 = 1
        } else {
            return A * pangkat(A, B - 1); // Rekursi: A^B = A * A^(B-1)
        }
    }

    public static void main(String[] args) {
        int A = 2;
        int B = 5;
        System.out.println(A + "^" + B + " = " + pangkat(A, B)); // Output: 2^5 = 32
    }
}

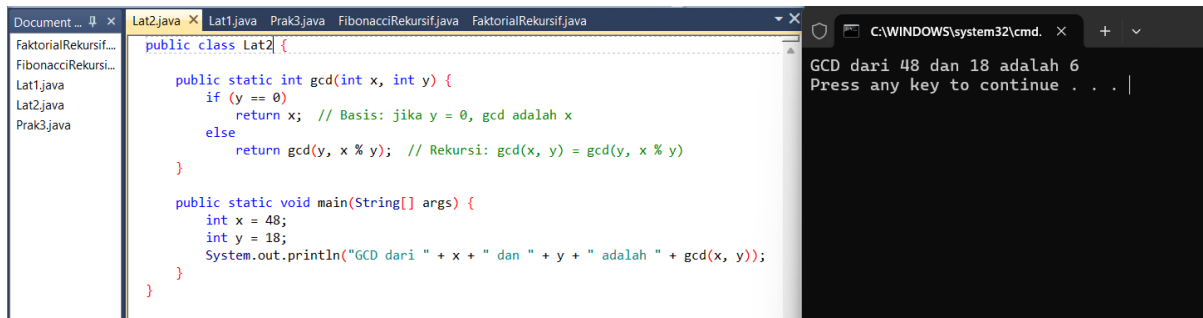
C:\WINDOWS\system32\cmd. x + v

2^5 = 32
Press any key to continue . . . |
```

Pembahasan : Program PangkatRekursif mendefinisikan method pangkat yang menerima dua parameter int A dan int B. Method ini memanfaatkan basis rekursi if ($B == 0$) untuk

mengembalikan nilai 1 karena pangkat nol dari bilangan manapun adalah 1. Langkah rekursinya adalah $\text{return } A * \text{pangkat}(A, B - 1)$ yang berarti bahwa A dipangkatkan dengan B didefinisikan sebagai A dikalikan dengan hasil dari A dipangkatkan B-1. Pada method main, dipanggil $\text{pangkat}(2, 5)$ untuk menghitung 2^5 yang hasilnya 32.

Latihan 2



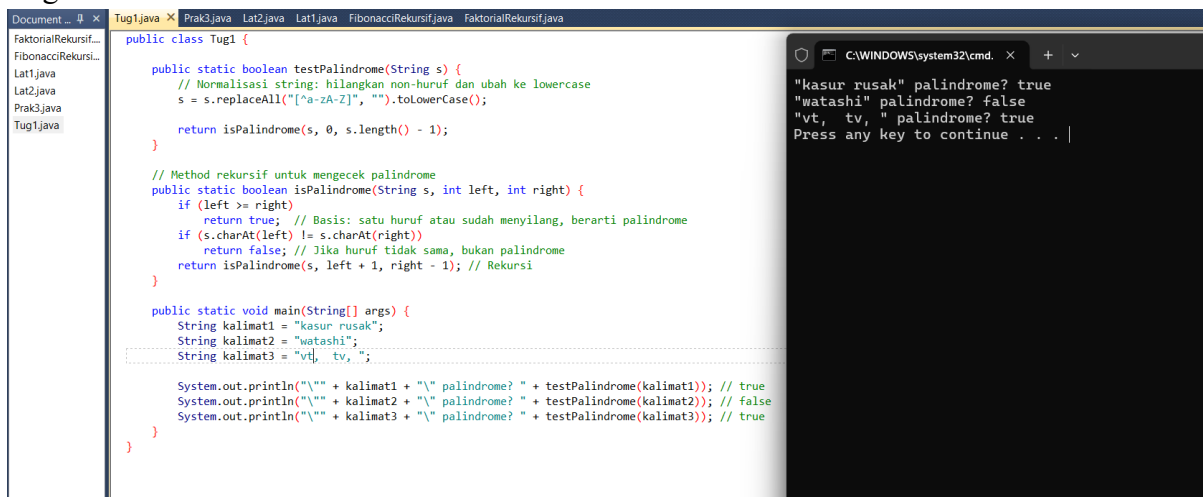
```
public class Lat2 {  
    public static int gcd(int x, int y) {  
        if (y == 0)  
            return x; // Basis: jika y = 0, gcd adalah x  
        else  
            return gcd(y, x % y); // Rekursi: gcd(x, y) = gcd(y, x % y)  
    }  
  
    public static void main(String[] args) {  
        int x = 48;  
        int y = 18;  
        System.out.println("GCD dari " + x + " dan " + y + " adalah " + gcd(x, y));  
    }  
}
```

GCD dari 48 dan 18 adalah 6
Press any key to continue . . . |

Pembahasan : Pada program GCDRekursif, didefinisikan method gcd yang menerima dua parameter int x dan int y. Basis rekursi adalah $\text{if } (y == 0)$ mengembalikan x karena jika salah satu angka nol, GCD-nya adalah angka lainnya. Langkah rekursinya adalah $\text{return gcd}(y, x \% y)$, yang sesuai dengan algoritma Euclidean untuk mencari pembagi bersama terbesar. Pada method main, dipanggil $\text{gcd}(48, 18)$ dan hasilnya adalah 6.

D. PEMBAHASAN TUGAS

Tugas 1



```
public class Tug1 {  
    public static boolean testPalindrome(String s) {  
        // Normalisasi string: hilangkan non-huruf dan ubah ke lowercase  
        s = s.replaceAll("[^a-zA-Z]", "").toLowerCase();  
  
        return isPalindrome(s, 0, s.length() - 1);  
    }  
  
    // Method rekursif untuk mengecek palindrome  
    public static boolean isPalindrome(String s, int left, int right) {  
        if (left >= right)  
            return true; // Basis: satu huruf atau sudah menyalang, berarti palindrome  
        if (s.charAt(left) != s.charAt(right))  
            return false; // Jika huruf tidak sama, bukan palindrome  
        return isPalindrome(s, left + 1, right - 1); // Rekursi  
    }  
  
    public static void main(String[] args) {  
        String kalimat1 = "kasur rusak";  
        String kalimat2 = "watashi";  
        String kalimat3 = "vt, tv,";  
  
        System.out.println("'" + kalimat1 + "' palindrome? " + testPalindrome(kalimat1)); // true  
        System.out.println("'" + kalimat2 + "' palindrome? " + testPalindrome(kalimat2)); // false  
        System.out.println("'" + kalimat3 + "' palindrome? " + testPalindrome(kalimat3)); // true  
    }  
}
```

"kasur rusak" palindrome? true
"watashi" palindrome? false
"vt, tv," palindrome? true
Press any key to continue . . . |

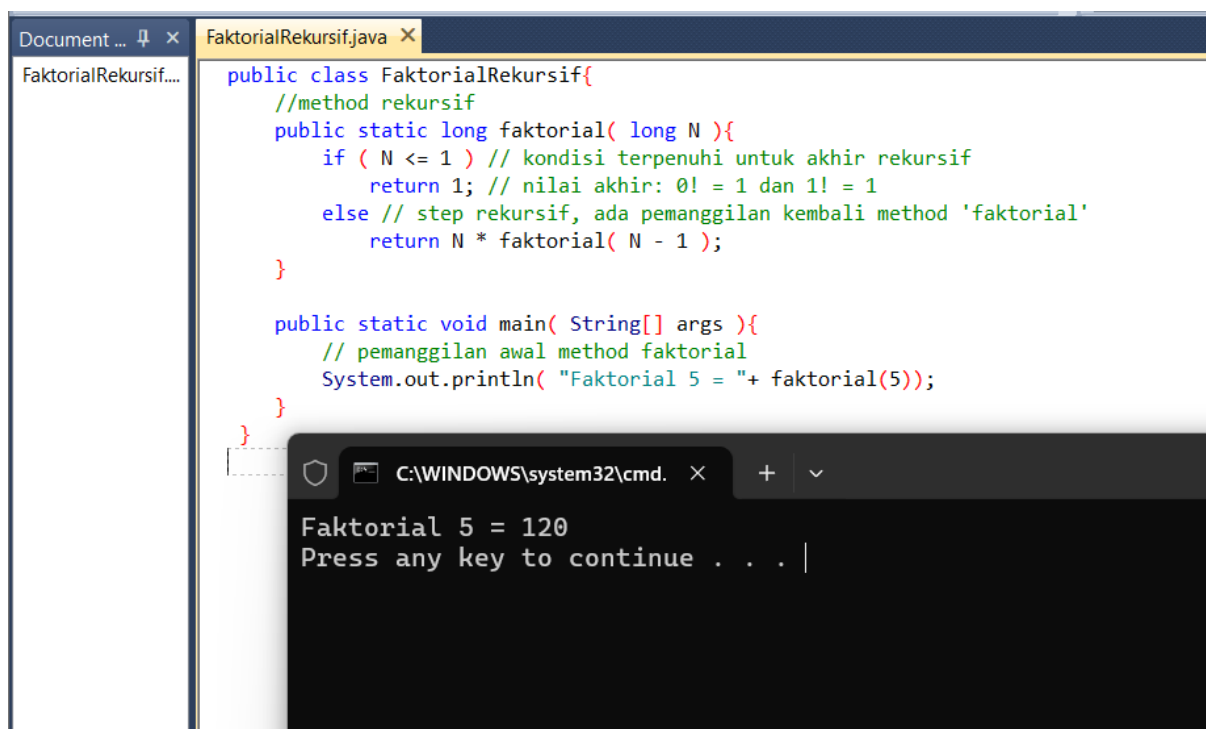
Pembahasan : Program PalindromeRekursif memiliki method testPalindrome yang bertipe boolean dan menerima parameter String s. Pertama-tama, string dinormalisasi dengan menghapus semua karakter selain huruf dan mengubahnya menjadi huruf kecil menggunakan `replaceAll("[^a-zA-Z]", "")` dan `toLowerCase()`. Kemudian, method isPalindrome dipanggil dengan parameter string tersebut, indeks kiri 0, dan indeks kanan `s.length() - 1`. Method

isPalindrome adalah method rekursif dengan basis rekursi if ($\text{left} \geq \text{right}$) yang akan mengembalikan true, artinya semua pasangan karakter telah cocok. Jika $\text{s.charAt(left)} \neq \text{s.charAt(right)}$, maka return false karena string bukan palindrom. Jika cocok, maka rekursi berlanjut dengan memanggil dirinya sendiri dengan $\text{left} + 1$ dan $\text{right} - 1$. Method ini diuji dengan tiga string, dan hasilnya menunjukkan apakah string tersebut palindrom atau bukan.

E. KESIMPULAN

rekursif memungkinkan sebuah method memanggil dirinya sendiri untuk menyelesaikan permasalahan yang lebih kecil hingga mencapai kondisi dasar (base case). Pada praktik faktorial, deret Fibonacci, pangkat, dan GCD, semua contoh tersebut menerapkan prinsip rekursi yang terdiri dari basis rekursi dan langkah rekursi (recursive step). Basis rekursi berfungsi sebagai kondisi berhenti agar pemanggilan tidak berlangsung terus menerus tanpa akhir, sedangkan langkah rekursi membuat method terus memecah masalah menjadi versi yang lebih sederhana. Dalam tugas pengecekan palindrome, rekursi digunakan untuk membandingkan karakter dari ujung ke ujung secara berpasangan

F. LAMPIRAN



```
Document ... x FaktorialRekursif.java x
FaktorialRekursif... public class FaktorialRekursif{
    //method rekursif
    public static long faktorial( long N ){
        if ( N <= 1 ) // kondisi terpenuhi untuk akhir rekursif
            return 1; // nilai akhir: 0! = 1 dan 1! = 1
        else // step rekursif, ada pemanggilan kembali method 'faktorial'
            return N * faktorial( N - 1 );
    }

    public static void main( String[] args ){
        // pemanggilan awal method faktorial
        System.out.println( "Faktorial 5 = "+ faktorial(5));
    }
}
```

```
C:\WINDOWS\system32\cmd. x + v
Faktorial 5 = 120
Press any key to continue . . . |
```

```
Document ... 1 x FibonacciRekursif.java x FaktorialRekursif.java
FaktorialRekursif...
FibonacciRekursi...

public class FibonacciRekursif{
    //method rekursif
    public static long fibonacci( long N ){
        if (( N == 0 ) || ( N == 1 )) // kondisi terpenuhi untuk akhir rekursif
            return N; // nilai akhir : N=0 atau N=1
        else // step rekursif, ada pemanggilan kembali method 'fibonacci'
            return fibonacci(N-1) + fibonacci(N-2);
    }

    public static void main( String[] args ){
        System.out.println("Deret fibonacci dengan N = 3");
        for (int i=0; i<=3;i++)
            // pemanggilan awal method fibonacci
            if (i<3)
                System.out.print(fibonacci(i)+", ");
            else
                System.out.println(fibonacci(i));
    }
}

C:\WINDOWS\system32\cmd. x + v
Deret fibonacci dengan N = 3
0, 1, 1, 2
Press any key to continue . . . |
```

```
Document ... 1 x Prak3.java x FibonacciRekursif.java x FaktorialRekursif.java
FaktorialRekursif...
FibonacciRekursi...
Prak3.java

public class Prak3 {
    // Method rekursif untuk menghitung faktorial
    public static long faktorial(int n) {
        if (n == 1)
            return 1;
        else
            return n * faktorial(n - 1);
    }

    public static void main(String[] args) {
        int N = 5;
        System.out.println("Faktorial dari 1 sampai " + N + ":");

        for (int i = 1; i <= N; i++) {
            long hasil = faktorial(i);
            System.out.println(i + "! = " + hasil);
        }
    }
}

C:\WINDOWS\system32\cmd. x + v
Faktorial dari 1 sampai 5:
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
Press any key to continue . . . |
```



```
Document ... x
Prak3.java x FibonacciRekursif.java FaktorialRekursif.java
FaktorialRekursif...
FibonacciRekursi...
Prak3.java

public class Prak3 {

    // Method rekursif untuk menghitung faktorial
    public static long faktorial(int n) {
        if (n == 1)
            return 1;
        else
            return n * faktorial(n - 1);
    }

    public static void main(String[] args) {
        int N = 5;
        System.out.println("Faktorial dari 1 sampai " + N + ":");

        for (int i = 1; i <= N; i++) {
            long hasil = faktorial(i);
            System.out.println(i + "! = " + hasil);
        }
    }
}
```

```
C:\WINDOWS\system32\cmd. x
Faktorial dari 1 sampai 5:
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
Press any key to continue . . . |
```

```
Document ... x
Lat1.java x Prak3.java FibonacciRekursif.java FaktorialRekursif.java
FaktorialRekursif...
FibonacciRekursi...
Lat1.java
Prak3.java

public class Lat1 {

    public static int pangkat(int A, int B) {
        if (B == 0) {
            return 1; // Basis: A^0 = 1
        } else {
            return A * pangkat(A, B - 1); // Rekursi: A^B = A * A^(B-1)
        }
    }

    public static void main(String[] args) {
        int A = 2;
        int B = 5;
        System.out.println(A + "^" + B + " = " + pangkat(A, B)); // Output: 2^5 = 32
    }
}
```

```
C:\WINDOWS\system32\cmd. x
2^5 = 32
Press any key to continue . . . |
```

```
Document ... x
Lat2.java x Lat1.java Prak3.java FibonacciRekursif.java FaktorialRekursif.java
FaktorialRekursif...
FibonacciRekursi...
Lat1.java
Lat2.java
Prak3.java

public class Lat2 {

    public static int gcd(int x, int y) {
        if (y == 0)
            return x; // Basis: jika y = 0, gcd adalah x
        else
            return gcd(y, x % y); // Rekursi: gcd(x, y) = gcd(y, x % y)
    }

    public static void main(String[] args) {
        int x = 48;
        int y = 18;
        System.out.println("GCD dari " + x + " dan " + y + " adalah " + gcd(x, y));
    }
}
```

```
C:\WINDOWS\system32\cmd. x
GCD dari 48 dan 18 adalah 6
Press any key to continue . . . |
```

```
Document ... x
Tug1.java x Prak3.java Lat2.java Lat1.java FibonacciRekursif.java FaktorialRekursif.java
FaktorialRekursif...
FibonacciRekursi...
Lat1.java
Lat2.java
Prak3.java
Tug1.java

public class Tug1 {

    public static boolean testPalindrome(String s) {
        // Normalisasi string: hilangkan non-huruf dan ubah ke lowercase
        s = s.replaceAll("[^a-zA-Z]", "").toLowerCase();

        return isPalindrome(s, 0, s.length() - 1);
    }

    // Method rekursif untuk mengecek palindrome
    public static boolean isPalindrome(String s, int left, int right) {
        if (left >= right)
            return true; // Basis: satu huruf atau sudah menyilang, berarti palindrome
        if (s.charAt(left) != s.charAt(right))
            return false; // Jika huruf tidak sama, bukan palindrome
        return isPalindrome(s, left + 1, right - 1); // Rekursi
    }

    public static void main(String[] args) {
        String kalimat1 = "kasur rusak";
        String kalimat2 = "watashi";
        String kalimat3 = "vtv tv";

        System.out.println("\n" + kalimat1 + "\n" + "palindrome? " + testPalindrome(kalimat1)); // true
        System.out.println("\n" + kalimat2 + "\n" + "palindrome? " + testPalindrome(kalimat2)); // false
        System.out.println("\n" + kalimat3 + "\n" + "palindrome? " + testPalindrome(kalimat3)); // true
    }
}
```

```
C:\WINDOWS\system32\cmd. x
"kasur rusak" palindrome? true
"watashi" palindrome? false
"vtv tv" palindrome? true
Press any key to continue . . . |
```